
Foundations of Deep Learning

Lecture 04

Optimization Landscape of Neural Networks

Aurelien Lucchi

Fall 2026

Discuss properties of the loss surface of deep neural networks.

How do they depend on the type of architecture, including parameters such as width, depth, activation functions, etc?

Definition 1 (Global minimum)

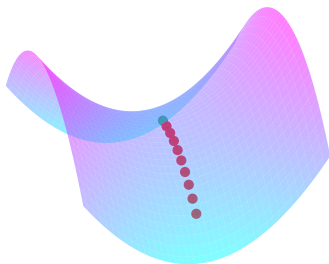
Given a function $f(\mathbf{w}) : \mathbb{R}^d \rightarrow \mathbb{R}$, a point $\mathbf{w}^*$ is called a global minimum of f if for every $\mathbf{w}$, we have $f(\mathbf{w}^*) \leq f(\mathbf{w})$.

Definition 2 (Local minimum)

Given a function $f(\mathbf{w}) : \mathbb{R}^d \rightarrow \mathbb{R}$, a point $\mathbf{w}^*$ is called a local minimum of f if there exists a neighborhood of size ϵ , i.e. $\{\mathbf{w} \in \mathbb{R}^d \mid \|\mathbf{w} - \mathbf{w}^*\| \leq \epsilon\}$ such that $f(\mathbf{w}^*) \leq f(\mathbf{w})$.

Definition 3 (Saddle point)

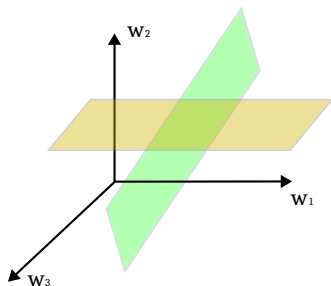
Given a function $f(\mathbf{w}) : \mathbb{R}^d \rightarrow \mathbb{R}$, a point $\mathbf{w}_s$ is called a saddle point of f if $\nabla f(\mathbf{w}_s) = 0$ and the Hessian $\nabla^2 f(\mathbf{w}_s)$ is indefinite, i.e. it has both positive and negative eigenvalues.



Note on Over-parametrization

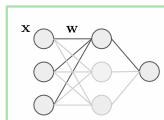
Consider a linear system of the form $F(\mathbf{w}) = \mathbf{y}$ where $\mathbf{w} \in \mathbb{R}^d$, $\mathbf{y} \in \mathbb{R}^n$ and $F : \mathbb{R}^d \rightarrow \mathbb{R}^n$.

As an example, take $n = 2, d = 3$.

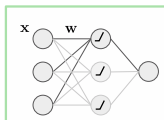


This is an over-parametrized model ($n < d$). There is a manifold of solutions corresponding to the intersection of the planes.

Landscapes of Neural Networks: Overview



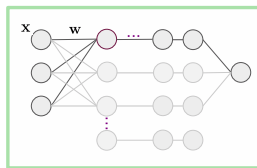
A) Linear network



B) Two-layer ReLU network



C) Midly-overparametrized network



D) Over-parametrized network/NTK regime

- ▶ A) Linear networks (*Kawaguchi (2016)*): Every local minima is global.
- ▶ B) Small-size two-layer ReLU networks (*Safran and Shamir (2018)*; *Yun et al. (2018)*): there exist spurious local minima and there is a high probability of reaching them
- ▶ D) Over-parametrized networks (*Du et al. (2018)*; *Allen-Zhu et al. (2018)*): every local minima is global

SECTION 0.1

Deep Linear Gradients

Gradient of the matrix Frobenius norm: For a matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$:

$$\nabla_{\mathbf{X}} \|\mathbf{X}\|_F^2 = 2\mathbf{X}$$

Derivation:

$$\nabla_{x_{ij}} \|\mathbf{X}\|_F^2 = \nabla_{x_{ij}} \left(\sum_{i=1}^m \sum_{j=1}^n x_{ij}^2 \right) = 2x_{ij}.$$

Gradient of the trace of a matrix product:

$$\begin{aligned}\nabla_{\mathbf{X}} \text{tr}(\mathbf{A}\mathbf{X}) &= \mathbf{A}^\top, \\ \nabla_{\mathbf{X}} \text{tr}(\mathbf{A}\mathbf{X}^\top) &= \mathbf{A}.\end{aligned}$$

Gradient of the quadratic form (symmetric $\mathbf{A}$): For a symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and vector $\mathbf{x} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A}\mathbf{x}$$

Gradient of the quadratic form (non-symmetric $\mathbf{A}$): For a non-symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and vector $\mathbf{x} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}$$

Gradient of the quadratic form with respect to $\mathbf{A}$: For a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and vector $\mathbf{x} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{A}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = \mathbf{x}\mathbf{x}^\top$$

Gradient of $\mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x}$: For a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and vector $\mathbf{x} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A}^\top \mathbf{A} \mathbf{x}$$

Gradient of $(\mathbf{y} - \mathbf{A} \mathbf{x})^\top (\mathbf{y} - \mathbf{A} \mathbf{x})$ with respect to $\mathbf{A}$:

$$\frac{\partial}{\partial \mathbf{A}} (\mathbf{y} - \mathbf{A} \mathbf{x})^\top (\mathbf{y} - \mathbf{A} \mathbf{x}) = -2(\mathbf{y} - \mathbf{A} \mathbf{x}) \mathbf{x}^\top.$$

Network Architecture: The input-output map is defined as:

$$\mathbf{y} = \mathbf{A}\mathbf{B}\mathbf{x}$$

where $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{y} \in \mathbb{R}^k$, $\mathbf{B} \in \mathbb{R}^{m \times d}$, $\mathbf{A} \in \mathbb{R}^{k \times m}$.

Training Objective: Given a set of n training examples $(\mathbf{x}_i, \mathbf{y}_i)_{i=1}^n$, the loss function is:

$$\mathcal{L}(\mathbf{A}, \mathbf{B}) = \frac{1}{2} \sum_{i=1}^n \|\mathbf{y}_i - \mathbf{A}\mathbf{B}\mathbf{x}_i\|^2.$$

Gradient of the loss with respect to B:

$$\frac{\partial \mathcal{L}(\mathbf{A}, \mathbf{B})}{\partial \mathbf{B}} = -\mathbf{A}^\top (\Sigma_{\mathbf{y}\mathbf{x}} - \mathbf{A}\mathbf{B}\Sigma_{\mathbf{x}\mathbf{x}}),$$

where $\Sigma_{\mathbf{x}\mathbf{x}}$ and $\Sigma_{\mathbf{y}\mathbf{x}}$ are input and input-output correlation matrices.

In order to calculate the gradients, let's first expand the above expression:

$$\begin{aligned}\mathcal{L}(\mathbf{A}, \mathbf{B}) &= \frac{1}{2} \sum_{i=1}^n \|\mathbf{y}_i - \mathbf{A}\mathbf{B}\mathbf{x}_i\|^2 \\&= \frac{1}{2} \sum_{i=1}^n (\mathbf{y}_i - \mathbf{A}\mathbf{B}\mathbf{x}_i)^\top (\mathbf{y}_i - \mathbf{A}\mathbf{B}\mathbf{x}_i) \\&= \frac{1}{2} \sum_{i=1}^n \mathbf{y}_i^\top \mathbf{y}_i - \mathbf{y}_i^\top \mathbf{A}\mathbf{B}\mathbf{x}_i - (\mathbf{A}\mathbf{B}\mathbf{x}_i)^\top \mathbf{y}_i + (\mathbf{A}\mathbf{B}\mathbf{x}_i)^\top (\mathbf{A}\mathbf{B}\mathbf{x}_i) \\&= \frac{1}{2} \sum_{i=1}^n \mathbf{y}_i^\top \mathbf{y}_i - \mathbf{y}_i^\top \mathbf{A}\mathbf{B}\mathbf{x}_i - \mathbf{x}_i^\top \mathbf{B}^\top \mathbf{A}^\top \mathbf{y}_i + \mathbf{x}_i^\top \mathbf{B}^\top \mathbf{A}^\top \mathbf{A}\mathbf{B}\mathbf{x}_i.\end{aligned}$$

Recall the following equalities:

$$\frac{\partial \mathbf{a}^\top \mathbf{X} \mathbf{b}}{\partial \mathbf{X}} = \mathbf{a} \mathbf{b}^\top$$

$$\frac{\partial \mathbf{a}^\top \mathbf{X}^\top \mathbf{D} \mathbf{X} \mathbf{b}}{\partial \mathbf{X}} = \mathbf{D}^\top \mathbf{X} \mathbf{a} \mathbf{b}^\top + \mathbf{D} \mathbf{X} \mathbf{b} \mathbf{a}^\top,$$

therefore we have

$$\frac{\partial \mathbf{y}_i^\top \mathbf{A} \mathbf{B} \mathbf{x}_i}{\partial \mathbf{B}} = \mathbf{A}^\top \mathbf{y}_i \mathbf{x}_i^\top$$

$$\frac{\partial \mathbf{x}_i^\top \mathbf{B}^\top \mathbf{A}^\top \mathbf{y}_i}{\partial \mathbf{B}} = \mathbf{x}_i \mathbf{y}_i^\top \mathbf{A}$$

$$\frac{\partial \mathbf{x}_i^\top \mathbf{B}^\top \mathbf{A}^\top \mathbf{A} \mathbf{B} \mathbf{x}_i}{\partial \mathbf{B}} = \mathbf{A} \mathbf{A}^\top \mathbf{B} \mathbf{x}_i \mathbf{x}_i^\top + \mathbf{A}^\top \mathbf{A} \mathbf{B} \mathbf{x}_i \mathbf{x}_i^\top$$

$$= 2 \mathbf{A} \mathbf{A}^\top \mathbf{B} \mathbf{x}_i \mathbf{x}_i^\top.$$

Note that the first two derivatives are equal (since $\mathbf{y}_i^\top \mathbf{A} \mathbf{B} \mathbf{x}_i = \mathbf{x}_i^\top \mathbf{B}^\top \mathbf{A}^\top \mathbf{y}_i$), thus

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{A}, \mathbf{B})}{\partial \mathbf{B}} &= - \sum_{i=1}^n \mathbf{A}^\top \mathbf{y}_i \mathbf{x}_i^\top + \sum_{i=1}^n \mathbf{A}^\top \mathbf{A} \mathbf{B} \mathbf{x}_i \mathbf{x}_i^\top \\ &= -\mathbf{A}^\top (\Sigma_{\mathbf{y}\mathbf{x}} - \mathbf{A} \mathbf{B} \Sigma_{\mathbf{x}\mathbf{x}}), \end{aligned}$$

where $\Sigma_{\mathbf{x}\mathbf{x}} = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top$ is the input correlation matrix and $\Sigma_{\mathbf{y}\mathbf{x}} = \sum_{i=1}^n \mathbf{y}_i \mathbf{x}_i^\top$ is the input-output correlation matrix.

Alternative option using the matrix form

Let $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]$ and the output data matrix $\mathbf{Y} = [\mathbf{y}_1 \dots \mathbf{y}_n]$.
Then

$$\begin{aligned}\mathcal{L}(\mathbf{W}) &= \|\mathbf{Y} - \mathbf{ABX}\|_F^2 \\ &= \text{tr}[(\mathbf{Y} - \mathbf{ABX})(\mathbf{Y} - \mathbf{ABX})^\top].\end{aligned}$$

Use the following known expressions:

$$\begin{aligned}\frac{\partial}{\partial \mathbf{B}} \text{tr}[\mathbf{ABC}] &= \mathbf{A}^\top \mathbf{C}^\top \\ \frac{\partial}{\partial \mathbf{B}} \text{tr}[\mathbf{CB}^\top \mathbf{A}^\top] &= \mathbf{A}^\top \mathbf{C} \\ \frac{\partial}{\partial \mathbf{B}} \text{tr}[\mathbf{ABCB}^\top \mathbf{A}^\top] &= \mathbf{A}^\top \mathbf{ABC}^\top + \mathbf{A}^\top \mathbf{ABC}.\end{aligned}$$

Gradient of the loss with respect to $\mathbf{A}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}} = -(\Sigma_{\mathbf{y}\mathbf{x}} - \mathbf{A}\mathbf{B}\Sigma_{\mathbf{x}\mathbf{x}})\mathbf{B}^\top.$$

SECTION 2

Landscape of Deep Linear Networks

- ▶ Consider a linear network with two layers whose weights are defined by matrices $\mathbf{A} \in \mathbb{R}^{d \times p}$ and $\mathbf{B} \in \mathbb{R}^{p \times d}$.
- ▶ Assume $\mathbf{x}_i, \mathbf{y}_i \in \mathbb{R}^d$.
- ▶ Network has one hidden layer with p ($p \leq d$) units, and one output layer with d units.

Let $\mathbf{W} = \mathbf{AB}$, and define the following loss function,

$$\mathcal{L}(\mathbf{W}) = \sum_{i=1}^n \|\mathbf{y}_i - \mathbf{ABx}_i\|^2. \quad (1)$$

Define the following covariance matrices, $\Sigma_{\mathbf{x}\mathbf{x}} = \sum_i \mathbf{x}_i \mathbf{x}_i^\top$, $\Sigma_{\mathbf{x}\mathbf{y}} = \sum_i \mathbf{x}_i \mathbf{y}_i^\top$ and $\Sigma_{\mathbf{y}\mathbf{y}} = \sum_i \mathbf{y}_i \mathbf{y}_i^\top$.

Matrix form Let $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]$ and the output data matrix $\mathbf{Y} = [\mathbf{y}_1 \dots \mathbf{y}_n]$. Then

$$\begin{aligned}\mathcal{L}(\mathbf{W}) &= \|\mathbf{Y} - \mathbf{A}\mathbf{B}\mathbf{X}\|_F^2 \\ &= \text{tr}[(\mathbf{Y} - \mathbf{A}\mathbf{B}\mathbf{X})(\mathbf{Y} - \mathbf{A}\mathbf{B}\mathbf{X})^\top].\end{aligned}$$

Theorem 4

For any fixed $d \times p$ matrix $\mathbf{A}$ the function $\mathcal{L}(\mathbf{A}, \mathbf{B})$ is convex in the coefficients of $\mathbf{B}$ and attains its minimum for any $\mathbf{B}$ satisfying the equation

$$\mathbf{A}^\top \mathbf{A} \mathbf{B} \Sigma_{\mathbf{x}\mathbf{x}} = \mathbf{A}^\top \Sigma_{\mathbf{y}\mathbf{x}}. \quad (2)$$

Theorem 5

For any fixed $p \times d$ matrix $\mathbf{B}$ the function $\mathcal{L}(\mathbf{A}, \mathbf{B})$ is convex in the coefficients of $\mathbf{A}$ and attains its minimum for any $\mathbf{A}$ satisfying the equation

$$\mathbf{A}\mathbf{B}\Sigma_{\mathbf{x}\mathbf{x}}\mathbf{B}^\top = \Sigma_{\mathbf{y}\mathbf{x}}\mathbf{B}^\top. \quad (3)$$

Proof.

Left as an exercise.



Theorem 6

Assume that $\Sigma_{\mathbf{xx}}$ is invertible. If two matrices $\mathbf{A}$ and $\mathbf{B}$ define a critical point of $\mathcal{L}$ (i.e., a point where $\frac{\partial \mathcal{L}}{\partial a_{ij}} = \frac{\partial \mathcal{L}}{\partial b_{ij}} = 0$) then the global map $\mathbf{W} = \mathbf{AB}$ is of the form

$$\mathbf{W} = P_{\mathbf{A}} \Sigma_{\mathbf{xy}} \Sigma_{\mathbf{xx}}^{-1}, \quad (4)$$

where $P_{\mathbf{A}}$ the matrix of the orthogonal projection onto the subspace spanned by the columns of $\mathbf{A}$.

Theorem 7

If Σ_{xx} and Σ_{xy} are full rank and $\Sigma = \Sigma_{yx}\Sigma_{xx}^{-1}\Sigma_{xy}$ is full rank, then any local minimum is global and other critical points are saddle points.

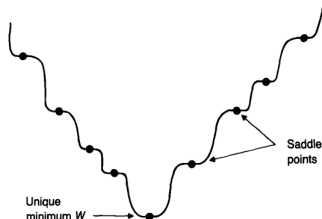


Figure: The landscape of the objective function (figure taken from [Baldi and Hornik(1989)]).

Consider the model $\hat{\mathbf{Y}} = \mathbf{W}_{H+1}\mathbf{W}_H \dots \mathbf{W}_1\mathbf{X}$ and the loss $\mathcal{L}(\mathbf{W}) = \frac{1}{2}\|\hat{\mathbf{y}}(\mathbf{W}, \mathbf{X}) - \mathbf{Y}\|^2$ and denote by p the smallest width in the hidden layers.

Theorem 8

For any depth $H \geq 1$ and for any layer widths and any input-output dimensions, the loss surface has the following properties:

- 1. It is non-convex and non-concave*
- 2. Every local minimum is a global minimum*
- 3. Every critical point that is not a global minimum is a saddle point*
- 4. If $\text{rank}(\mathbf{W}_H\mathbf{W}_{H-1} \dots \mathbf{W}_2) \geq p$, the Hessian at any saddle point has at least one negative eigenvalue.*

Deep linear networks provide a tractable setting in which the optimization landscape can be characterized exactly.

↪ Excludes non-linear neural networks.

The **Neural Tangent Kernel (NTK)** provides a framework for studying the training dynamics of nonlinear networks.

Main idea Linearize the network around initialization. At infinite width, this approximation remains accurate during training, and the network behaves like a kernel method.

↪ This allows us to study:

- ▶ optimization convergence;
- ▶ architecture and initialization;
- ▶ predictions and generalization.

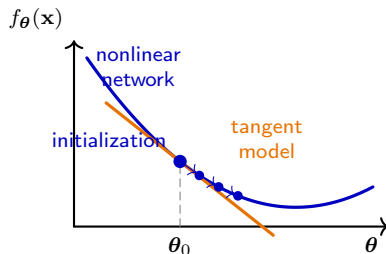
Linearizing a Neural Network

Let $f_{\theta}(\mathbf{x}) \in \mathbb{R}$ denote a neural network and let θ_0 be its initialization. Around initialization,

$$f_{\theta}(\mathbf{x}) \approx f_{\theta_0}(\mathbf{x}) + \langle \nabla_{\theta} f_{\theta_0}(\mathbf{x}), \theta - \theta_0 \rangle.$$

Define the **gradient features**

$$\phi_0(\mathbf{x}) = \nabla_{\theta} f_{\theta_0}(\mathbf{x}).$$



The network is nonlinear in its parameters, but its first-order approximation is linear in the displacement $\theta - \theta_0$.

For sufficiently wide networks, the parameters move little during training, so this linearization can remain accurate.

NTK For training inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$, define

$$K_{\theta}(i, j) = \langle \nabla_{\theta} f_{\theta}(\mathbf{x}_i), \nabla_{\theta} f_{\theta}(\mathbf{x}_j) \rangle.$$

For the squared loss

$$\mathcal{L}(\theta) = \frac{1}{2} \|\mathbf{f}_{\theta} - \mathbf{y}\|^2,$$

gradient flow gives

$$\frac{d\mathbf{f}_{\theta}}{dt} = -K_{\theta}(\mathbf{f}_{\theta} - \mathbf{y}).$$

If the kernel remains constant, then

$$\mathbf{f}_t - \mathbf{y} = e^{-K_0 t}(\mathbf{f}_0 - \mathbf{y}).$$

At a suitable random initialization, as the width tends to infinity,

$$K_{\theta(t)} \approx K_{\theta_0} \longrightarrow K_{\infty},$$

where K_{∞} is deterministic (*Jacot et al., 2018*).

Theorem 9

If K_{∞} is positive definite on the training data, gradient flow converges to zero training error. Thus, a non-convex neural-network problem behaves like convex kernel regression in function space.

- ▶ Wide networks have a locally benign optimization landscape near initialization.
- ▶ This is a **lazy-training** regime: the learned features change little.
- ▶ At finite width, the NTK can evolve; this feature learning is not captured by the fixed-kernel model.
- ▶ Fast convergence on the training set does not by itself guarantee good generalization.

SECTION 4

Vanishing and Exploding Gradients

Consider scalar numbers $a_1, a_2, \dots, a_T$ and define their product

$$P_T = \prod_{t=1}^T a_t.$$

1. If all $a_t < 1$ (in magnitude): **vanishing case**. For instance, let $a_t = 0.9$ for all t . Then

$$P_T = 0.9^T = e^{T \log 0.9} = e^{-0.105 T}.$$

$\leadsto$ quantity decays exponentially fast.

2. If all $a_t > 1$: **exploding case**. Similarly, if $a_t = 1.1$, then

$$P_T = 1.1^T = e^{T \log 1.1} = e^{0.095 T}.$$

$\leadsto$ product grows exponentially with T .

Consider the function $f(\mathbf{x}) = \mathbf{A}\mathbf{x} \in \mathbb{R}^p$, where $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{A} \in \mathbb{R}^{p \times d}$. Our goal is to compute the gradient $\frac{df}{d\mathbf{x}}$.

First, note that the dimension of the gradient $\frac{df}{d\mathbf{x}}$ is $\mathbb{R}^{p \times d}$. Let's compute the partial derivative of f w.r.t. a single x_j . We have

$$f_i(\mathbf{x}) = \sum_{j=1}^d A_{ij}x_j \implies \frac{\partial f_i}{\partial x_j} = A_{ij}. \quad (5)$$

Collecting all the partial derivatives in the Jacobian, we obtain the following expression for the gradient:

$$\frac{df}{d\mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_p}{\partial x_1} & \cdots & \frac{\partial f_p}{\partial x_d} \end{pmatrix} = \begin{pmatrix} A_{11} & \cdots & A_{1d} \\ \vdots & \ddots & \vdots \\ A_{p1} & \cdots & A_{pd} \end{pmatrix} = \mathbf{A} \in \mathbb{R}^{p \times d}. \quad (6)$$

Consider the function $f(\mathbf{x}) = \mathbf{A}\mathbf{x} \in \mathbb{R}^p$, where $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{A} \in \mathbb{R}^{p \times d}$. Our goal is to compute the gradient $\frac{df}{d\mathbf{A}}$.

First, note that the dimension of the gradient $\nabla f := \frac{df}{d\mathbf{A}}$ is $\mathbb{R}^{p \times (p \times d)}$. Let's compute the partial derivative of f w.r.t. a single x_j . We have

$$f_i(\mathbf{x}) = \sum_{j=1}^d A_{ij}x_j \implies \frac{\partial f_i}{\partial A_{iq}} = x_q. \quad (7)$$

Collecting all the partial derivatives, we can compute partial derivative of f_i w.r.t. the i -th row of $\mathbf{A}$:

$$\begin{aligned} \frac{\partial f_i}{\partial A_{i,:}} &= \mathbf{x}^\top \in \mathbb{R}^{1 \times 1 \times d} \\ \frac{\partial f_i}{\partial A_{:,i}} &= \mathbf{0}^\top \in \mathbb{R}^{1 \times 1 \times d}. \end{aligned} \quad (8)$$

Stacking the partial derivatives, we obtain the gradient of f_i w.r.t. $\mathbf{A}$:

$$\frac{\partial f_i}{\partial \mathbf{A}} = \begin{pmatrix} \mathbf{0}^\top \\ \dots \\ \mathbf{0}^\top \\ \mathbf{x}^\top \\ \mathbf{0}^\top \\ \dots \\ \mathbf{0}^\top \end{pmatrix} \in \mathbb{R}^{1 \times (p \times d)}. \quad (9)$$

One can use the Kronecker product notation $\otimes$ to write the total derivative of f w.r.t. $\mathbf{A}$ as

$$\frac{\partial f}{\partial \mathbf{A}} = \mathbf{x}^\top \otimes \mathbf{I}. \quad (10)$$

Consider a regression problem with a single datapoint $\mathbf{x} \in \mathbb{R}^d$ and a corresponding target $\mathbf{y} \in \mathbb{R}^d$.

Deep Linear Network

$$\hat{\mathbf{y}} := F(\mathbf{x}) = \mathbf{W}^{L:1} \mathbf{x}, \quad \mathbf{W}^{L:1} = \mathbf{W}^L \dots \mathbf{W}^1, \quad \mathbf{W}^k \in \mathbb{R}^{d \times d}$$

Interested in the case of random weight matrices, e.g. at initialization.

Squared Loss Given a single target $\mathbf{y}$,

$$\ell(\hat{\mathbf{y}}) = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}\|^2, \quad \frac{\partial \ell}{\partial \hat{\mathbf{y}}} = \hat{\mathbf{y}} - \mathbf{y} = \mathbf{W}^{L:1} \mathbf{x} - \mathbf{y} =: \boldsymbol{\delta}$$

$$\begin{aligned}\frac{\partial \ell}{\partial \mathbf{W}^k} &= \overbrace{\left[\mathbf{W}^{k+1:L} \boldsymbol{\delta} \right]}^{\text{backward}} \cdot \overbrace{\left[\mathbf{W}^{k-1:1} \mathbf{x} \right]^\top}^{\text{forward}} \\ &= \mathbf{W}^{k+1:L} [\mathbf{W}^{L:1} \mathbf{x} \mathbf{x}^\top - \mathbf{y} \mathbf{x}^\top] \mathbf{W}^{1:k-1},\end{aligned}$$

with $\mathbf{W}^{k+1:L} := (\mathbf{W}^{k+1})^\top \dots (\mathbf{W}^L)^\top$.

Theorem 10

Let $\mathbf{W}^k$ be Gaussian matrices with iid entries such that $\mathbb{E}[w_{ij}] = 0$ and $\mathbb{E}[w_{ij}^2] = \sigma^2$, and let $\rho = \|\mathbf{x}\|^4$, $\gamma = \|\mathbf{x}\|^2 \|\mathbf{y}\|^2$. Then:

$$\mathbb{E} \left\| \frac{\partial \ell}{\partial \mathbf{W}^k} \right\|_F^2 \leq 2 \frac{\rho}{3\sigma^2} [\sigma^4 d(d+2)]^L + 2\gamma (\sigma^2 d)^{L-1}.$$

Note that the exponents of the two contributions differ.

- ▶ Small σ : gradient is dominated by the term involving $\mathbf{y}$
- ▶ Large $\sigma^4 > d(d+2)$: term involving $\hat{\mathbf{y}}$ dominates (assuming $\rho \approx \gamma$)

Xavier initialization [Glorot and Bengio(2010)]

When d is very large, then the dominating term in the theorem is of the form $(\sigma^4 d^2)^L \implies$ Stabilization requires $\sigma = \frac{1}{\sqrt{d}}$.

Starting point We bound the norm of the gradient as follows,

$$\begin{aligned} \left\| \frac{\partial \ell}{\partial \mathbf{W}^k} \right\|_F^2 &= \left\| \mathbf{W}^{k+1:L} \mathbf{W}^{L:1} \mathbf{x} \mathbf{x}^\top \mathbf{W}^{1:k-1} - \mathbf{W}^{k+1:L} \mathbf{y} \mathbf{x}^\top \mathbf{W}^{1:k-1} \right\|_F^2 \\ &\leq 2 \left\| \mathbf{W}^{k+1:L} \mathbf{W}^{L:1} \mathbf{x} \mathbf{x}^\top \mathbf{W}^{1:k-1} \right\|_F^2 \\ &\quad + 2 \left\| \mathbf{W}^{k+1:L} \mathbf{y} \mathbf{x}^\top \mathbf{W}^{1:k-1} \right\|_F^2. \end{aligned}$$

Next, we bound each term independently.

Lemma 11

Let $\mathbf{W}^k$ be random matrices with iid entries such that $\mathbb{E}[w_{ij}] = 0$ and $\mathbb{E}[w_{ij}^2] = \sigma^2$. For a fixed input/output pair $(\mathbf{x}, \mathbf{y})$ one has

$$\begin{aligned} \mathbb{E} \left\| \frac{\partial \ell}{\partial \mathbf{W}^k} \right\|_F^2 &\leq 2\sigma^2 \underbrace{\mathbb{E} \|\mathbf{W}^{k-1:1} \mathbf{x}\|_2^4}_{\textcircled{A}: \text{Lemma 14}} \underbrace{\mathbb{E} \|\mathbf{W}^{k+1:L} \mathbf{W}^{L:k+1}\|_F^2}_{\textcircled{B}: \text{Lemma 12}} \\ &\quad + 2(d\sigma^2)^{L-1} \|\mathbf{x}\|^2 \|\mathbf{y}\|^2. \end{aligned}$$

We start with a simple lemma that we will apply recursively...

Lemma 12

Let $\mathbf{W}$ be a random matrix with iid entries such that $\mathbb{E}[w_{ij}] = 0$ and $\mathbb{E}[w_{ij}^2] = \sigma^2$, $\mathbf{A}, \mathbf{B}$ arbitrary matrices, then

$$\mathbb{E}\|\mathbf{AWB}\|_F^2 = \sigma^2 \|\mathbf{A}\|_F^2 \|\mathbf{B}\|_F^2.$$

Moreover, if $\mathbf{A}, \mathbf{B}$ are stochastic but $\{\mathbf{A}, \mathbf{B}, \mathbf{W}\}$ uncorrelated, then

$$\mathbb{E}\|\mathbf{AWB}\|_F^2 = \sigma^2 \cdot \mathbb{E}\|\mathbf{A}\|_F^2 \cdot \mathbb{E}\|\mathbf{B}\|_F^2.$$

By applying Lemma 12 recursively, we obtain the following:

Corollary 13

$$\mathbb{E}\|\mathbf{W}^{t:1}\mathbf{x}\|^2 = (d\sigma^2)^t\|\mathbf{x}\|^2$$

Lemma 14 (Statistics after multiplication with a random matrix)

Let $\mathbf{W}$ be a random matrix with iid entries such that $\mathbb{E}[w_{ij}] = 0$ and $\mathbb{E}[w_{ij}^2] = \sigma^2$, and kurtosis κ . Let $\boldsymbol{\xi} \in \mathbb{R}^d$ be an arbitrary random vector. Then

$$\mathbb{E}\|\mathbf{W}\boldsymbol{\xi}\|_2^4 = d(d+2)\sigma^4\mathbb{E}\|\boldsymbol{\xi}\|_2^4 + (\kappa - 3)d\sigma^4\mathbb{E}\|\boldsymbol{\xi}\|_4^4.$$

$$\begin{aligned}\|\mathbf{W}\boldsymbol{\xi}\|_2^4 &= \left(\sum_i \left(\sum_r w_{ir} \xi_r \right)^2 \right)^2 \\ &= \sum_{i,j} \sum_r w_{ir} \xi_r \sum_s w_{is} \xi_s \sum_u w_{ju} \xi_u \sum_v w_{jv} \xi_v.\end{aligned}$$

Then take an expectation...

Let $\mathbf{W}^k$ be Gaussian matrices with iid entries such that $\mathbb{E}[w_{ij}] = 0$ and $\mathbb{E}[w_{ij}^2] = \sigma^2$, and let $\rho = \|\mathbf{x}\|^4$, $\gamma = \|\mathbf{x}\|^2 \|\mathbf{y}\|^2$. Then:

$$\mathbb{E} \left\| \frac{\partial \ell}{\partial \mathbf{W}^k} \right\|_F^2 \leq 2 \frac{\rho}{3\sigma^2} [\sigma^4 d(d+2)]^L + 2\gamma(\sigma^2 d)^{L-1}.$$



Pierre Baldi and Kurt Hornik.

Neural networks and principal component analysis: Learning from examples without local minima.

Neural networks, 2(1):53–58, 1989.



Xavier Glorot and Yoshua Bengio.

Understanding the difficulty of training deep feedforward neural networks.

In **Proceedings of the thirteenth international conference on artificial intelligence and statistics**, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.



Kenji Kawaguchi.

Deep learning without poor local minima.

In **Advances in Neural Information Processing Systems**, pages 586–594, 2016.